Azure Service Bus – SQL Filters

1. What is a SQL Filter?

A **SQL Filter** in Azure Service Bus is used to filter messages based on a SQL-like expression applied to message properties.

For example:

```
City = 'Mumbai'
```

This means:

> The subscription will receive only those messages whose application property `City` has the value `Mumbai`.

---

2. Why Do We Use SQL Filters?

A topic can have multiple subscriptions, and each subscription can have its own filter.

For example:

```
                Topic
             OrderTopic
                 |
      ----------------------
      |          |          |
     S1         S2         S3
  Mumbai    Bangalore    Delhi
```

Filters:

```
S1 → City = 'Mumbai'
S2 → City = 'Bangalore'
S3 → City = 'Delhi'
```

When a message arrives, Azure Service Bus evaluates the filter for each subscription.

Only the subscriptions whose filter condition evaluates to **true** receive the message.

3. Example Scenario

We create:

```
Topic:
FirstTopic

Subscription:
S1

SQL Filter:
City = 'Mumbai'
```

Now we send a message with this application property:

```
City = Mumbai
```

The filter evaluates:

```
City = 'Mumbai'
```

Result:

```
TRUE
```

Therefore:

```
Message → S1 ✅
```

4. SQL Filters Use SQL-Like Expressions

SQL filters support SQL-like expressions and operators.

For example:

```
City = 'Mumbai'
```

Other examples include:

```
OrderAmount > 10000
```

```
Priority = 'High'
```

```
City = 'Mumbai' AND OrderAmount > 10000
```

```
City = 'Mumbai' OR City = 'Delhi'
```

So SQL filters are useful when the filtering requirement is more complex than simple equality.

---

5. Important: String Values Need Quotes

When comparing a string property, use quotes around the value.

Correct:

```
City = 'Mumbai'
```

Incorrect:

```
City = Mumbai
```

For example:

```
CustomerType = 'Premium'
```

But for numeric values:

```
OrderAmount > 10000
```

we don't use quotes around the number.

6. Create the Service Bus Administration Client

The first step is to create the administration client:

```
var adminClient =
    new ServiceBusAdministrationClient(connectionString);
```

The administration client is used to manage Service Bus entities such as:

- Topics
- Subscriptions
- Rules
- Filters

7. Create Subscription S1

Create the subscription options:

```
var s1Details =
    new CreateSubscriptionOptions(
        topicName,
        "S1");
```

Here:

```
topicName → Topic where subscription will be created

S1 → Subscription name
```

8. Create SQL Filter Rule

Now create the SQL filter rule:

```
var sqlRule =
    new CreateRuleOptions(
```

```
        "SQLRule",
        new SqlRuleFilter("City = 'Mumbai'"));
```

There are two important parts:

Rule Name

```
SQLRule
```

Filter Condition

```
City = 'Mumbai'
```

This means:

> Check the `City` application property and accept the message when its value is `Mumbai`.

---

9. Create the Subscription

Finally, create the subscription:

```
await adminClient.CreateSubscriptionAsync(
    s1Details,
    sqlRule);
```

After execution, the structure becomes:

```
Topic
  |
  └── S1
       |
       └── SQLRule
            |
            └── SQL Filter
                 City = 'Mumbai'
```

---

10. Complete Subscription Creation Code

A simplified example:

```
var adminClient =
    new ServiceBusAdministrationClient(connectionString);

var s1Details =
    new CreateSubscriptionOptions(
        topicName,
        "S1");

var sqlRule =
    new CreateRuleOptions(
        "SQLRule",
        new SqlRuleFilter("City = 'Mumbai'"));

await adminClient.CreateSubscriptionAsync(
    s1Details,
    sqlRule);
```

---

11. Verify the Filter in Azure Portal

After running the application:

```
Azure Portal
    ↓
Service Bus Namespace
    ↓
Topic
    ↓
Subscriptions
    ↓
S1
```

Open subscription `S1`.

You should see:

```
Rule:
SQLRule
```

```
Filter Type:
SQL Filter

Condition:
City = 'Mumbai'
```

This confirms that the SQL filter has been created successfully.

---

12. Sending a Message with the Required Property

Creating the filter alone is not enough.

The message must contain the property used by the filter.

For example:

```
var message =
    new ServiceBusMessage("Hello Service Bus");

message.ApplicationProperties.Add(
    "City",
    "Mumbai");
```

Now the message contains:

```
Message Body:
Hello Service Bus

Application Property:
City = Mumbai
```

---

13. Send the Message

Send the message to the topic:

```
await sender.SendMessageAsync(message);
```

The flow is:

```
C# Application
      ↓
Create Message
      ↓
Add Application Property
City = Mumbai
      ↓
Send Message
      ↓
Topic
      ↓
Subscription S1
      ↓
SQL Filter
City = 'Mumbai'
      ↓
MATCH
      ↓
Message delivered to S1
```

---

14. How SQL Filter Evaluates the Message

Suppose the incoming message has:

```
Application Property:

City = Mumbai
```

The SQL filter is:

```
City = 'Mumbai'
```

Service Bus evaluates:

```
Message City = Mumbai
Filter Value = Mumbai
```

Therefore:

```
Mumbai = Mumbai
       ↓
     TRUE
```

The message is delivered to S1.

---

15. What If City = Bangalore?

Suppose the message contains:

```
City = Bangalore
```

But the filter is:

```
City = 'Mumbai'
```

The evaluation becomes:

```
Bangalore = Mumbai
        ↓
      FALSE
```

Therefore:

```
Message → S1 ❌
```

The message is not delivered to S1.

---

16. What If the City Property Is Missing?

Suppose the message doesn't contain:

```
City
```

The SQL filter expects:

```
City = 'Mumbai'
```

Since the required property is not available, the message does not satisfy the filter.

Therefore:

```
Message → S1 ❌
```

---

17. SQL Filter vs Correlation Filter

This is an important interview question.

Correlation Filter

Correlation filters are mainly used for simple equality matching.

Example:

```
City = Mumbai
```

Conceptually:

```
Property → Value
```

SQL Filter

SQL filters use SQL-like expressions.

Example:

```
City = 'Mumbai'
```

They can also be used for more complex conditions.

Example:

```
City = 'Mumbai' AND OrderAmount > 10000
```

Another example:

```
City = 'Mumbai' OR City = 'Delhi'
```

---

18. Simple Comparison

Feature	Correlation Filter	SQL Filter
Simple equality	Yes	Yes
SQL-like expressions	No	Yes
Multiple conditions	Limited	Yes
AND / OR conditions	No	Yes
Greater than / less than	No	Yes
Best for	Simple matching	Complex filtering

Easy Way to Remember

```
Correlation Filter
        ↓
Simple equality

SQL Filter
        ↓
SQL-like conditions
```

---

19. Real-Time Example

Imagine an e-commerce system.

We have:

```
OrderTopic
```

Subscriptions:

```
MumbaiOrders
HighValueOrders
PremiumOrders
```

Possible SQL filters:

```
MumbaiOrders
→ City = 'Mumbai'
```

```
HighValueOrders
→ OrderAmount > 50000
```

```
PremiumOrders
→ CustomerType = 'Premium'
```

Suppose the message has:

```
OrderId = 101
City = Mumbai
OrderAmount = 75000
CustomerType = Premium
```

The message can match multiple subscriptions:

```
MumbaiOrders
City = 'Mumbai'
       ↓
     TRUE ✅

HighValueOrders
OrderAmount > 50000
       ↓
     TRUE ✅

PremiumOrders
CustomerType = 'Premium'
```

```
      ↓
TRUE ✅
```

Therefore, the same message can be delivered to multiple subscriptions.

---

20. More SQL Filter Examples

Equality

```
City = 'Mumbai'
```

Not Equal

```
City <> 'Mumbai'
```

Greater Than

```
OrderAmount > 10000
```

Less Than

```
OrderAmount < 10000
```

Greater Than or Equal

```
OrderAmount >= 10000
```

Less Than or Equal

```
OrderAmount <= 10000
```

AND

```
City = 'Mumbai' AND OrderAmount > 10000
```

Both conditions must be true.

OR

```
City = 'Mumbai' OR City = 'Delhi'
```

At least one condition must be true.

---

21. Important Concept – Body vs Application Property

Just like correlation filters, SQL filters work with **message properties**.

For example, the message body could be:

```
{
  "Name": "Viral",
  "City": "Mumbai"
}
```

And the application property is:

```
City = Mumbai
```

The SQL filter:

```
City = 'Mumbai'
```

checks the **message/application property**.

It does not simply search inside the JSON body.

Therefore, if you want to filter based on `City`, add it as an application property:

```
message.ApplicationProperties.Add(
    "City",
    "Mumbai");
```

---

22. Testing Using Service Bus Explorer

After creating the SQL filter, open:

```
Azure Portal
    ↓
Service Bus
    ↓
Topic
    ↓
Service Bus Explorer
```

Choose **Send messages**.

Add the custom property:

```
Key:
City

Value:
Mumbai
```

Then send the message.

Go to subscription `S1`.

You should see:

```
Message Count = 1
```

Open the message.

You should see:

```
Message Body:
Hello...

Message Properties:
City = Mumbai
```

This confirms that the SQL filter matched the message.

23. Complete End-to-End Flow

```
Step 1
Create Topic
        ↓
Step 2
Create Subscription S1
        ↓
Step 3
Create SQL Filter
City = 'Mumbai'
        ↓
Step 4
Create Service Bus Message
        ↓
Step 5
Add Application Property
City = Mumbai
        ↓
Step 6
Send Message to Topic
        ↓
Step 7
Service Bus evaluates SQL filter
        ↓
City = 'Mumbai' ?
        ↓
       YES
        ↓
Message delivered to S1
```

24. Interview Questions

Q1. What is a SQL Filter in Azure Service Bus?

A SQL Filter is used to filter messages using SQL-like expressions on message properties.

Example:

```
City = 'Mumbai'
```

Only messages satisfying this condition are delivered to the subscription.

Q2. Where do we define the SQL Filter?

The filter is defined on a **subscription of a Service Bus topic**.

```
Topic
  ↓
Subscription
  ↓
SQL Filter
```

Q3. Can SQL filters support multiple conditions?

Yes.

For example:

```
City = 'Mumbai' AND OrderAmount > 10000
```

Q4. What is the difference between SQL Filter and Correlation Filter?

A correlation filter is mainly intended for simple equality-based matching, while a SQL filter supports SQL-like expressions and more complex conditions.

Q5. Does the SQL filter search inside the JSON message body?

No.

The filter evaluates message properties. If we want to filter using a custom property such as `City`, we should add it as an application property.

Example:

```
message.ApplicationProperties.Add(
    "City",
    "Mumbai");
```

Q6. Can one message be delivered to multiple subscriptions?

Yes.

If the message satisfies the filters of multiple subscriptions, it can be delivered to all matching subscriptions.

Example:

```
Message
   ↓
S1 → City = 'Mumbai'        ✅
S2 → OrderAmount > 50000    ✅
S3 → City = 'Delhi'         ❌
```

The message is delivered to S1 and S2.

---

25. Key Points for Quick Revision

- **SQL Filter** = SQL-like filtering of Service Bus messages.
- It is configured on a **subscription**.
- It works with **message/application properties**.
- String values should be enclosed in quotes.
- Example:

```
City = 'Mumbai'
```

- Supports operators and logical conditions such as:

```
=
<>
>
<
>=
<=
```

```
AND
OR
```

- SQL filters are useful for **complex filtering requirements**.
- A message must contain the property required by the filter.
- If the filter evaluates to `true`, the subscription receives the message.
- If the filter evaluates to `false`, that subscription does not receive the message.

---

26. One-Line Interview Answer

> **A SQL filter in Azure Service Bus is a subscription-level filter that uses SQL-like expressions to evaluate message properties and determine whether a message should be delivered to that subscription.**

Easy Memory Trick

```
Correlation Filter
→ Simple equality

SQL Filter
→ SQL-like expression
→ AND / OR
→ > / < / >= / <=
→ Complex filtering
```